

SiNPlE : Simplified Inference of Novel Polymorphisms from Large coverage

SiNPlE is a fast and sensitive variant caller based on a simplified Bayesian approach to compute the posterior probability that a variant is not generated by sequencing errors or PCR artefacts. It is particularly suited to low-frequency variant calling, as described in [Ferretti et al. \(2019\)](#). While calling heterozygous variants in samples with defined ploidy is relatively straightforward — for instance, the expected allele frequency is 50% or 100% in the diploid case — the underlying assumptions for low-frequency variant calling are very different and require a different statistical approach. In the Bayesian model used by SiNPlE we explicitly consider the limit in which ploidy tends to infinity, which is equivalent to a mixture with a very large number of molecules, and thus no artificial constraints are imposed. You can still use SiNPlE if you know what the ploidy of your system is — you just need to post-filter the calls made by SiNPlE.

Comparison with other callers

Some variant callers, such as GATK, assume a ploidy of two. Others have hidden hardwired thresholds — for instance, a minimum number of covering reads needed to make a call — and hence miss many existing low-frequency variants. A detailed comparison with the low-frequency variant callers LoFreq and VarScan2 performed on CirSeq data can be found in [Ferretti et al., 2019](#); it shows that the approach adopted by SiNPlE is faster and much more accurate.

Citation

If you use SiNPlE, please cite

Ferretti L, Tennakoon C, Silesian A, Freimanis G, Ribeca P. SiNPlE: Fast and Sensitive Variant Calling for Deep Sequencing Data. Genes. 2019; 10(8):561. <https://doi.org/10.3390/genes10080561>

Table of contents

- 1. Installation
 - 1.1. Conda channel
 - 1.2. Pre-compiled binaries
 - 1.3. Manual install
- 2. How to run it
- 3. Interpreting the output
 - 3.1. SiNPlE's own tabular output
 - 3.2. VCF output
- 4. Command line syntax

1. Installation

 Note that the only operating systems we officially support are Linux and MacOS. 

OCaml is highly portable and you might be able to manually compile/install everything successfully on other platforms (for instance, Windows) but you will have to do it yourself.

There are several possible ways of installing the software on your machine: through `conda`; by downloading pre-compiled binaries (Linux x86_64, and macOS on Apple Silicon or Intel); or manually.

1.1. Conda channel

 Coming soon! 

1.2. Pre-compiled binaries

Pre-compiled binaries for Linux (x86_64) and macOS (Apple Silicon and Intel) are published on our [releases](#) page. Download the archive for your platform, unpack it, and copy or move the `SiNPlE` binary it contains to a directory which is accessible from your `PATH`.

For instance, supposing that you've copied the binary to directory `~/local/bin/`, in order to make them accessible from everywhere you'll have to execute a command such as

```
export PATH=~/local/bin:$PATH
```

or add it to one of your login scripts (such as `~/bashrc` or similar for `bash`).

The Linux binaries are statically linked according to the recipe described [here](#) and have no runtime dependencies. The macOS binaries use the GMP library as installed by [Homebrew](#), so on a Mac type `brew install gmp` before running them.

1.3. Manual install

Alternatively, you can install `SiNPlE` manually by cloning and compiling its sources. You'll need an up-to-date distribution of the OCaml compiler and the [Dune package manager](#) for that, plus the OCaml packages `menhir` and `zarith` (the latter needs the GMP library). All of them can be installed through [OPAM](#), the official OCaml distribution system. Once you have a working OPAM distribution you'll also have a working OCaml compiler, and the rest can be installed with the command

```
$ opam install dune menhir zarith
```

if not already present. Make sure that you install OCaml version 4.14 or later.

`SiNPlE` is built on the [BiOCamLib library](#), which comes with it as a Git submodule, so clone the repository with `--recursive` to pull it in:

```
$ git clone --recursive git@github.com:PaoloRibeca/SiNPlE.git
```

The version the program reports is worked out from the history of the repository, so build from a full clone like the one above: a downloaded archive of the sources does not build, and a shallow clone (made with `--depth`) reports a wrong version. Then go to the directory into which you have cloned the sources, and type

```
$ bash BUILD release-static
```

(or `bash BUILD release` on macOS, where fully static linking is not possible). That leaves the executable in `.build/SiNPlE`; copy it to some favourite location in your `PATH`, for instance `~/local/bin`. If `samtools` is installed, the build also checks the new executable against recorded output (`test/characterize`).

2. How to run it

`SiNPlE` chooses its input format with `--input-format` (`mpileup` by default), its output format with `--output-format` (`vcf` by default), and reads from standard input and writes to standard output unless `-i` and `-o` name files.

The input is either the pileup format produced by commands such as `samtools mpileup`, or, with `--input-format gem`, the [GEM mapper](#)'s own output. The output is either a VCF, `SiNPlE`'s own table with `--output-format simple`, or both. A VCF to standard output is thus the shortest invocation:

```
$ samtools mpileup -f reference.fasta -d 1000000 -a -A -B -Q 0 -x example.bam | SiNPlE > variants.vcf
```

With GEM, no BAM or pileup is needed: `gem3-mapper -F MAP` writes every placement of every read on the read's line, and `SiNPlE` walks that against the reference the reads were mapped to (`-r`), reading what each read says about each position directly. The reads must come from FASTQ, as the model needs their qualities; `--strata` says how many of a read's best strata of placements count (1 by default, its best placements alone), every placement in them counting, so that a read on several copies of a repeat is evidence at each of them. The shorthand `--map <reference>` is `--input-format gem --reference <reference>` :

```
$ gem3-mapper -I reference.gem -i reads.fastq -M all -F MAP | SiNPlE --map reference.fasta > variants.vcf
```

`--output-format both` writes the table and the VCF together. Then `-o` is a prefix that gains `.sinple` and `.vcf` , unless it names a device under `/dev/` or is left unspecified, when the two are written to it as they are:

```
$ samtools mpileup -f reference.fasta -d 1000000 -a -A -B -Q 0 -x example.bam | SiNPlE --output-format both -o run
# writes run.sinple and run.vcf
```

3. Interpreting the output

3.1. SiNPlE 's own tabular output

The output generated by `SiNPlE` is made of tab-separated lines, one for each position of the sequence(s) being explored for which there is coverage in the pileup. The number of columns is variable, and equals $2 + 4 * n$, where n is the number of genotypes — i.e., the different symbols occurring in the pileup; they might be nucleotides such as `A` , `C` , `G` , `T` , `N` , or indels such as `+AAA` . An example record might be

```
My_precious Tab 12037 Tab A Tab 85760 Tab 36.7 Tab 1 Tab G Tab 61 Tab 35.3 Tab 0.0243
```

The first two columns,

```
My_precious Tab 12037
```

are sequence name and position. They are followed by groups of four columns, such as

```
A Tab 85760 Tab 36.7 Tab 1
```

which are the sequence of the genotype at that position (`A` in this case) with the number of reads it occurs in (85760), the average sequencing (*not* alignment) quality for such reads, and the posterior probability that that genotype is actually present in what you are sequencing (1 in this case).

In general there'll be more than one genotype in the output for each position (as many as seen in the pileup, in fact) possibly including the one present in the reference, which has no special status in the model used by `SiNPlE` . They are all considered independently, as the statistical hypothesis is that we are in the presence of a mixture of genotypes. In this case, at this position one also sees 61 `G` s, but, given the priors, the frequencies, and the qualities, the model thinks it is noise (posterior probability $p = 0.0243$). That is not always the case, and, as expected, sometimes you can have several genotypes whose presence at the same position is considered statistically significant by `SiNPlE` .

Typically you would keep genotypes that have, for instance, $p \geq 0.95$. The priors can be adjusted as needed, in particular the ones for indels.

3.2. VCF output

In the VCF generated by `SiNPlE` , each record is a site where some read said something other than the reference base, every such genotype being an alternate allele, anchored on the reference base as the format wants; the record carries the reads voting at the site and for the reference base, and for each alternate allele its reads, their fraction, their mean base quality and the

posterior, with one sample column (`--vcf-sample`) of allele depths. As in the table, nothing is left out: the threshold `--vcf-minimum-posterior` (0.95 by default) marks the records rather than choosing them, `FILTER` being `PASS` when some alternate allele has a posterior at or above it and `LowPosterior` otherwise. The reference base must be known, which it is with `--map` and with an mpileup made with `-f`.

4. Command line syntax

This is the full list of command line options available for the program `SiNPlE`. You can visualise the list by typing

```
$ SiNPlE -h
```

in your terminal. You will see a header containing information about the version:

```
This is SiNPlE version 1.2.1-159 [22-Sep-2026]
compiled against: BiOCamLib version 1.3.4-1057 [21-Sep-2026]
(c) 2017-2019 Luca Ferretti <luca.ferretti@gmail.com>
    2017-2019 Chandana Tennakoon <drcyber@gmail.com>
    2017-2026 Paolo Ribeca <paulo.ribeca@gmail.com>
```

followed by detailed information. The general form(s) the command can be used is:

```
SiNPlE [OPTIONS]
```

Algorithmic parameters

Option	Argument(s)	Effect	Note(s)
<code>-t</code> <code>--theta</code>	<i>non_negative_float</i>	prior estimate of nucleotide diversity	<u>default= 0.001</u>
<code>-T</code> <code>--theta-indel</code>	<i>non_negative_float</i>	prior estimate of indel likelihood	<u>default= 0.0001</u>
<code>-I</code> <code>--quality-indel-short</code>	<i>non_negative_integer</i>	prior Phred-scaled quality for indels of length 1	<u>default= 35</u>
<code>-L</code> <code>--quality-indel-long</code>	<i>non_negative_integer</i>	prior Phred-scaled quality for indels of length >1	<u>default= 45</u>
<code>-p</code> <code>--pcr-error-rate</code>	<i>non_negative_float</i>	prior estimate of error rate for PCR-generated substitutions	<u>default= 5e-07</u>
<code>-P</code> <code>--pcr-error-rate-indel</code>	<i>non_negative_float</i>	prior estimate of error rate for PCR-generated indels	<u>default= 5e-08</u>
<code>--error-rate</code>	<i>non_negative_float</i>	prior estimate of error rate for sequencing-generated substitutions	<u>default= 0.0001</u>
<code>--error-rate-indel-short</code>	<i>non_negative_float</i>	prior estimate of error rate for sequencing-generated indels of length 1	<u>default= 1e-05</u>
<code>--error-rate-indel-long</code>	<i>non_negative_float</i>	prior estimate of error rate for sequencing-generated indels of length >1	<u>default= 1e-06</u>
<code>-s</code> <code>-S</code> <code>--strandedness</code>	<i>forward reverse both</i>	strands to be taken into account for counts	<u>default= both</u>

Input

Option	Argument(s)	Effect	Note(s)
<code>-f</code> <code>--input-format</code>	<i>mpileup gem</i>	format of the input: an mpileup, as samtools mpileup writes it, or what gem3-mapper -F MAP wrote for the reads mapped to the reference given with -r, walked against it directly. The reads must have been mapped from FASTQ, as the model needs their qualities	default= <u><i>mpileup</i></u>
<code>-i</code> <code>--input</code>	<i>input_file</i>	name of the input file	default= <u><i><stdin></i></u>
<code>-r</code> <code>--reference</code>	<i>reference_fasta_file</i>	the reference the reads were mapped to, needed and used only when the input format is <i>gem</i>	default= <u><i><none></i></u>
<code>-m</code> <code>--map</code>	<i>reference_fasta_file</i>	shorthand for <code>--input-format gem --reference <reference_fasta_file></code>	
<code>--strata</code>	<i>positive_integer</i>	with the <i>gem</i> input format, count only the placements in the first that many non-empty strata of each read, a stratum being its placements with the same number of errors: 1 keeps its best alone	default= <u><i>1</i></u>

Output

Option	Argument(s)	Effect	Note(s)
<code>-F</code> <code>--output-format</code>	<i>single vcf both</i>	format of the output: SiNPle's own table, a VCF, or both. When both, -o is a prefix that gains <i>.single</i> and <i>.vcf</i> , unless it names a device under <i>/dev/</i> or is left unspecified, when both are written there as they are	default= <u><i>vcf</i></u>
<code>-o</code> <code>--output</code>	<i>output_file_or_prefix</i>	name of the output file, or its prefix when the output format is <i>both</i>	default= <u><i><stdout></i></u>
<code>--vcf-minimum-posterior</code>	<i>fraction</i>	in the VCF, the posterior some alternate allele needs for its record to PASS rather than be marked LowPosterior	default= <u><i>0.95</i></u>
<code>--vcf-sample</code>	<i>name</i>	the name of the VCF's one sample	default= <u><i>sample</i></u>

Miscellaneous

Option	Argument(s)	Effect	Note(s)
<code>-v</code> <code>--verbose</code>		set verbose execution	default= <u><i>false</i></u>
<code>-V</code> <code>--version</code>		print version and exit	
<code>-h</code> <code>--help</code>		print syntax and exit	